Volume 01

WarrantyGraph
Architecture & System Map

Layers, packages, APIs, topology

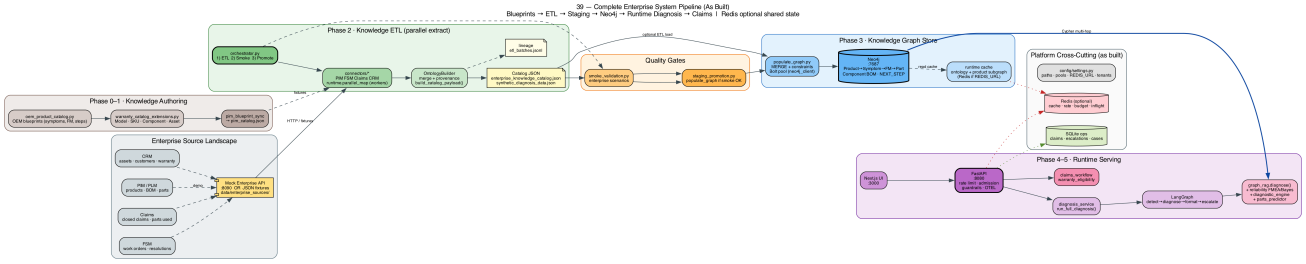
Full project library
This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. Problem & solution

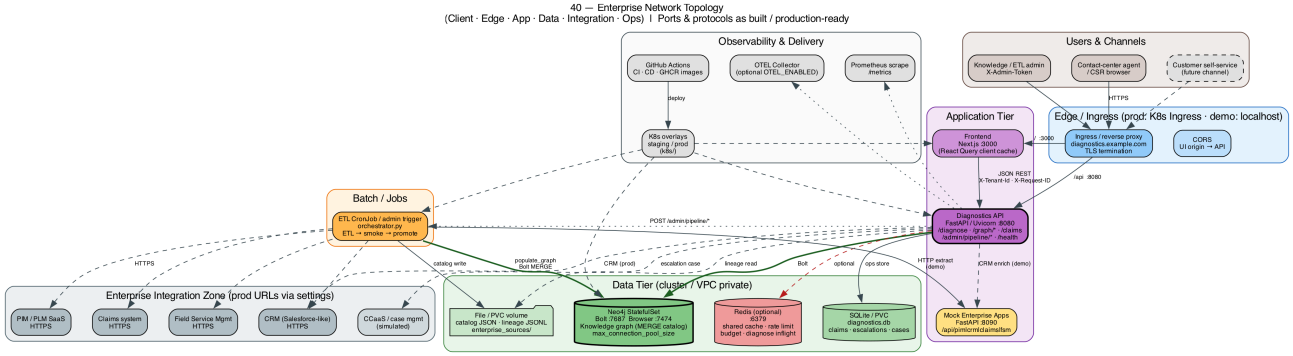
Warranty agents need consistent, explainable diagnosis: which failure mode, which part, is it covered? The system encodes OEM knowledge as a graph and serves multi-hop reasoning via GraphRAG — not free-form LLM invention.

2. Six layers

Layer	Packages	Job
L6 Experience	frontend/	Chat, explorer, claims, ops
L5 Orchestration	services/, agents/, integrations/	Warranty, LangGraph, claims
L4 Intelligence	graph_rag, reliability, parts, trees	Score & explain
L3 Integration	connectors, simulation	PIM/CRM/FSM/Claims
L2 Graph store	neo4j_client, populate_graph	MERGE + Cypher
L1 Data platform	ETL, OEM catalog, fixtures	Author & load knowledge



Complete pipeline



Network topology

3. Package map (entire repo)

- **api/** REST · **services/** business path · **agents/** LangGraph
- **graph/** intelligence + enterprise_pipeline ETL + OEM catalog
- **integrations/** CRM, warranty, claims, cases · **simulation/** mock :8090
- **runtime/** cache/Redis/concurrency · **guardrails/** safety
- **observability/** logs/metrics/traces · **gateway/** optional LLM
- **frontend/** Next.js · **evals/** + **tests/** quality
- **docker/ k8s/ deploy/ monitoring/ security/** production shape

4. HTTP API surface

Method	Path	Why it exists
POST	/diagnose	Primary customer diagnosis
GET	/graph/*	Explorer + schema + path subgraphs
/claims	Warranty claim lifecycle	
GET	/health /metrics	Ops readiness + Prometheus
/admin/pipeline/	ETL dry-run, validate, promote (gated)	

5. Online vs batch

```
ONLINE: UI → API → diagnosis_service → LangGraph → graph_rag → Neo4j

BATCH:  connectors → OntologyBuilder → catalog JSON → smoke → promote → MERGE
```

6. Core subsystems in What/Where/When/How/Why form

WHAT	POST /diagnose — customer diagnosis response
WHERE	api/main.py → services/diagnosis_service.py → agents + graph_rag
WHEN	Every agent/customer diagnosis request
HOW	Guardrails → CRM/warranty → LangGraph → Cypher + FMEA/Bayes → format
WHY	Single explainable path; no LLM required for facts

WHAT	Knowledge load into Neo4j
WHERE	populate_graph.py · orchestrator · staging_promotion
WHEN	Batch / admin promote / CLI — not per diagnose
HOW	create_constraints then MERGE nodes & relationships
WHY	Idempotent catalog materialization for multi-hop reads

WHAT	Neo4j unique indexes (via constraints)
WHERE	graph/populate_graph.py create_constraints
WHEN	Start of every populate_graph()
HOW	REQUIRE product_id, symptom_id, failure_mode_id, … IS UNIQUE
WHY	Index seek on MATCH {id:$x}; prevent duplicates

WHAT	Optional Redis shared state
WHERE	runtime/redis_client.py · rate_limit · cache · budget · concurrency_limit
WHEN	When REDIS_URL is set and Redis reachable; else memory fallback
HOW	Shared keys for multi-pod rate/cache/admission
WHY	Correct multi-replica behavior without sticky sessions